

😊 Add icon 🖼️ Add cover

Credit Decisioning Operating System

This is actually a much bigger opportunity than simply building a credit underwriting model.

What you're describing is essentially a **Credit Decisioning Operating System** where the ML scorecard becomes one of several decision engines inside an Agentic Underwriting Framework.

The problem most lenders face today is not the lack of data. It's that:

- Banking says turnover is ₹4 Cr
- GST says turnover is ₹5 Cr
- Financials say turnover is ₹6 Cr
- Bureau shows leverage inconsistent with both
- Sales proposes ₹50L
- Policy says max 20% of turnover
- Existing exposure says limit should be ₹25L

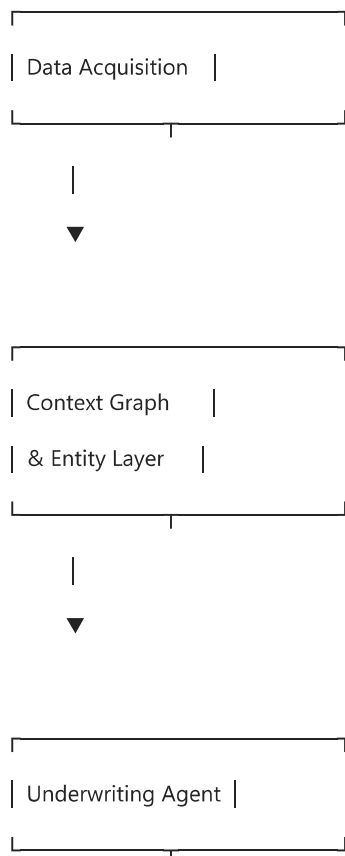
The underwriter's real job is not scoring.

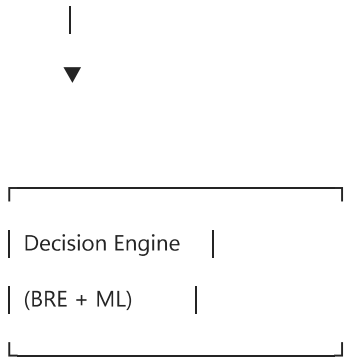
It's **triangulation**.

Proposed Architecture

Credit Intelligence Fabric

Think of the system as 4 layers.





Layer 1: Data Acquisition Agents

Each source becomes an independent agent.

Banking Agent

Inputs:

- Bank statements
- Account aggregator feeds
- Bureau banking feeds

Produces:

```
{
  "monthly_credit_turnover": 45000000,
  "avg_balance": 750000,
  "bounce_rate": 2.1,
  "cash_deposit_ratio": 15,
  "seasonality_score": 0.82,
  "stability_score": 0.71
}
```

GST Agent

Produces:

```
{
  "gst_turnover": 52000000,
  "filing_compliance": 96,
  "gstr1_gstr3b_match": 98,
  "input_output_ratio": 0.72,
  "gst_growth": 22
}
```

Bureau Agent

Produces:

```
{
  "commercial_score": 745,
  "director_score": 782,
  "current_exposure": 14000000,
  "delinquency_score": 0.12
}
```

Financial Analysis Agent

Produces:

```
{
  "revenue": 60000000,
  "ebitda_margin": 13,
  "debt_equity": 1.2,
  "dscr": 1.8
}
```

Ledger Agent

Produces:

```
{
  "debtor_days": 48,
  "supplier_concentration": 23,
  "anchor_concentration": 40,
  "invoice_velocity": 0.83
}
```

Layer 2: Context Graph

This becomes the most important component.

Instead of storing raw data.

Store relationships.

Example:

Customer

```
|
|
|—— GST Turnover
```

|

|— Banking Turnover

|

|— Financial Revenue

|

|— Ledger Sales

Now calculate:

GST ↔ Banking variance

Banking ↔ Financial variance

GST ↔ Ledger variance

This becomes a trust graph.

Example

GST Revenue:

₹5 Cr

Bank Revenue:

₹4.8 Cr

Financial Revenue:

₹9 Cr

Variance:

GST vs Bank = 4%

GST vs Financials = 80%

Financials become suspicious

Agent learns:

Trust GST

Trust Banking

Reduce trust in Financials

This is exactly what a human underwriter does.

Layer 3: Triangulation Agent

This becomes the brain.

It asks:

What is the true turnover?

Inputs:

GST = 5 Cr

Bank = 4.8 Cr

Financials = 9 Cr

Output:

Effective Turnover = 4.9 Cr

Confidence = 92%

What is working capital need?

Uses:

Debtor Days

Creditor Days

Inventory Cycle

Anchor Payment Cycle

Produces:

Working Capital Gap = ₹45L

What is repayment capacity?

Combines:

Bank cashflow

EBITDA

GST trend

Debt obligations

Produces:

Repayment Capacity = ₹52L

What is fraud probability?

Checks:

GST mismatch

Round tripping

Related party flows

Invoice concentration

Synthetic turnover

Produces:

Fraud Risk = Medium

Layer 4: BRE Agent

Current lenders already have:

DSCR > 1.25

GST Vintage > 12 months

Bureau > 700

Max DPD = 30

Anchor Concentration < 70%

Rather than replacing BRE.

Agent calls BRE.

bre.evaluate(customer)

Output:

```
{  
  "rule_pass_rate": 92,  
  "failed_rules": [  
    "GST Vintage"  
  ]  
}
```

Layer 5: ML Risk Agent

Your Balanced Scorecard lives here.

Input:

All engineered features

Output:

```
{  
  "risk_score": 83,  
  "pd": 1.8,  
  "expected_color": "Green"  
}
```

Layer 6: Sanction Optimization Agent

This is where most lenders stop today.

The real question is:

How much should I lend?

Not:

Should I lend?

Agent combines:

Constraint 1

Policy

Max 20% of turnover

→ ₹98L

Constraint 2

Working capital need

₹45L

Constraint 3

Repayment capacity

₹52L

Constraint 4

Risk appetite

Risk Score 83

Multiplier = 0.9

Constraint 5

Existing exposure

Current loans = ₹18L

Optimization engine:

Min(

Policy Limit,

WC Need,

Repayment Capacity,

Risk Appetite Limit

)

Result:

Recommended Limit = ₹45L

Final Credit Memo Agent

The final output should not be a score.

It should look like a credit manager's note.

Recommended SCF Limit: ₹45L

Confidence: 91%

Risk Score: 83 (Green)

Key Drivers:

- Banking and GST turnover aligned within 4%
- Strong repayment behavior
- Bureau score 745
- Working capital gap ₹45L

Key Risks:

- 40% anchor concentration
- One delayed GST filing

Policy Deviations:

- GST vintage short by 2 months

Recommendation:

Approve ₹45L with monthly monitoring.

Why this becomes a Plug-and-Play Lending OS

The architecture is product-agnostic.

For:

- SCF
- Invoice Finance
- OD
- Working Capital
- Term Loan
- Dealer Finance
- Vendor Finance

Only two things change:

1. Product Rules

Different BRE.

2. Optimization Constraints

Different sanction logic.

Everything else remains identical:

Data Agents

→ Context Graph

→ Triangulation Engine

→ Risk Model

→ BRE

→ Limit Optimizer

→ Credit Memo

This is the architecture I would recommend if the goal is to build a lender-grade underwriting copilot that can eventually automate 70–80% of MSME underwriting decisions while retaining full explainability for credit teams and regulators.

For a Wells Fargo-style demo, I would not build a "credit model demo."

I would build a **Lending Intelligence Operating System** that happens to contain underwriting.

That positioning is far more compelling because every large bank already has:

- Scorecards
- Decision engines
- LOS
- Fraud tools
- Bureau integrations
- Data warehouses

What they don't have is a system that can:

1. Collect evidence across disconnected systems
2. Reconstruct a business digitally
3. Detect contradictions automatically
4. Explain a decision with evidence
5. Generate a regulator-ready credit memo
6. Learn continuously from post-disbursement behavior

That is the gap.

Demo Vision

Imagine Wells Fargo uploads:

- Bank statements
- Financial statements
- GST/VAT equivalent
- Bureau report
- Supplier invoices
- ERP extracts
- KYC documents

Within 5 minutes the platform produces:

Company Intelligence Graph

Visual map of:

- Entity
- Directors
- Customers
- Suppliers
- Banking relationships
- Debt obligations

Underwriting View

Revenue Assessment

- Reported Revenue = \$10M
- Bank inferred revenue = \$8.7M
- Invoice inferred revenue = \$9.1M

Confidence = 82%

Contradictions

Detected Issues:

- ! Revenue mismatch
- ! Undisclosed liabilities
- ! Circular transactions
- ! Customer concentration

Risk Score

Risk Grade: B+

PD: 2.4%

Recommended Limit:

- Requested = \$1M
- Recommended = \$720k

Credit Memo

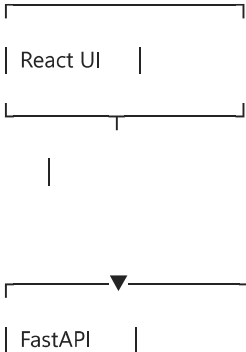
Auto-generated 6-page memo

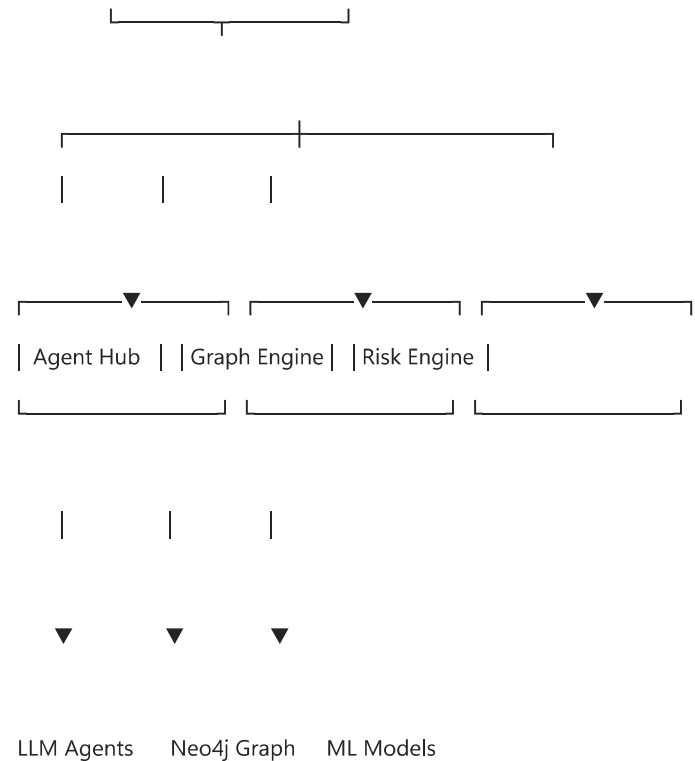
Exactly what a credit analyst would have written.

That demo will get attention.

Prototype Architecture

Build it entirely with open source.





Core Open Source Stack

□

	≡ Layer	≡ Technology
1	Frontend	React + NextJS
2	Backend	FastAPI
3	Workflow	LangGraph
4	Agents	CrewAI / PydanticAI
5	LLM	Llama 3.3 / Qwen 3
6	Vector DB	Qdrant
7	Graph DB	Neo4j
8	Feature Store	Feast
9	ML Models	XGBoost
10	Monitoring	Langfuse
11	Data Validation	Great Expectations
12	OCR	PaddleOCR

13	Document Parsing	Unstructured.io
14	Dashboard	Streamlit
15	Containerization	Docker

Everything above can be demonstrated locally.

Agent Framework

This is where most “agentic underwriting” ideas fail.

Most people create:

Document Agent

→ Risk Agent

→ Decision Agent

That becomes a hallucination machine.

Instead:

Evidence-First Architecture

Every agent must produce:

```
{
  "finding": "Revenue Concentration",
  "evidence": [
    "Invoice_34",
    "BankTxn_123",
    "GST_Record_9"
  ],
  "confidence": 0.91
}
```

No evidence.

No conclusion.

Agents To Build

1. KYC Agent

Extracts:

- Company
- Directors
- Ownership
- Registration

Output:

```
{  
  "directors":[]  
}
```

2. Banking Agent

Processes synthetic bank statements.

Calculates:

- Average balance
- Cash inflow
- EMI obligations
- Bounce ratio

3. Financial Agent

Reads P&L and Balance Sheet.

Calculates:

- EBITDA
- Debt service coverage
- Leverage

4. Fraud Agent

Identifies:

- Round tripping
- Layering
- Related-party patterns

5. Cash Flow Reconstruction Agent

This is the most valuable agent.

Large institutions struggle here.

The problem:

MSME books ≠ reality.

This agent rebuilds:

Actual Revenue

Actual Expenses

Actual Debt

Actual Working Capital

- Bank data
- GST
- Invoices
- Ledger

This is what I would emphasize to Wells Fargo.

Not AI.

Not agents.

Not scorecards.

Trust Graph

Build using Neo4j.

Company

|

├── Supplier

|

├── Customer

|

├── Director

|

├── Bank Account

|

└── Loan

Every data point becomes a node.

This allows:

- Relationship analysis
- Beneficial ownership
- Connected exposure
- Fraud discovery

Most underwriting platforms cannot do this.

Synthetic Data Strategy

Do NOT use real customer data.

Generate:

Companies

faker

Create:

- 500 MSMEs
- 100 suppliers
- 50 anchors

Banking

Generate:

- Salary credits
- GST payments
- EMI payments
- Vendor payouts

GST

Generate:

- Sales
- Purchases
- Tax filings

Bureau

Create synthetic:

- Credit score
- Enquiries
- Delinquencies

Risk Model

For demo purposes:

Use XGBoost.

Features:

- Bureau
- Banking
- GST
- Financials

Output:

Risk Score

0-1000

Map to:

Super Green

Green

Amber

Red

Exit

This aligns well with your existing SCF scorecard vision and dynamic post-sanction framework.

What To Show Wells Fargo**Scenario 1**

Healthy Business

Output:

Approve

Scenario 2

Revenue Inflation

Output:

Conditional Approval

Agent explains why.

Scenario 3

Fraud

Output:

Reject

Evidence shown.

Scenario 4

Existing Customer

Load monthly repayment data.

Show:

Initial Score = 720

Month 1 = 730

Month 2 = 755

Month 3 = 710

Demonstrates your dynamic post-sanction scoring model.

Competitive Positioning

Do not position against LOS vendors.

Do not position against decision engines.

Position against the manual underwriting operating model itself.

Tagline:

"A plug-and-play Lending Intelligence OS that reconstructs business reality from fragmented data, generates explainable underwriting decisions, and continuously learns from borrower behavior."

That is substantially more differentiated than "AI underwriting" and aligns with the problems large institutions still struggle to solve despite having mature lending stacks.